

Université des Sciences et de la Technologie Houari Boumediène
(USTHB)

RAPPORT DE MINI-PROJET MULTIMÉDIA

Pipeline d'Encodage Vidéo Simplifié de type MPEG-4



Enseignante :

- Nom et Prénom de l'Enseignant de TP Multimédia : *B.Abadli*

Réalisé par le binôme :

DELLACHE Hind
222231621615

BENCHABANE Sarah
212131107874

1re Année Maste IL

Année universitaire : 2025/2026

1. Introduction

La compression vidéo est au cœur de tous les systèmes multimédias modernes. Chaque vidéo visionnée en streaming, enregistrée ou partagée repose sur un codec — un système qui encode des données visuelles brutes en un flux binaire compact, puis les décode pour restituer les images d'origine.

Ce projet a pour objectif d'implémenter en Python un pipeline d'encodage vidéo simplifié, inspiré du standard MPEG-4. Il couvre l'ensemble du processus, depuis un dossier d'images séquentielles jusqu'à un fichier binaire compressé (.bin), décodable en images reconstruites.

Objectif du projet

Implémenter un encodeur/décodeur vidéo simplifié de type MPEG-4 en Python, couvrant : la conversion d'espace colorimétrique, la compression par DCT (I-frames), l'estimation de mouvement (P-frames), le codage entropique LZW et l'évaluation par métriques de qualité (PSNR, ratio de compression).

1.1 Paramètres globaux du projet

Paramètre	Valeur	Description
Q (facteur de quantification)	50	Qualité de compression (10=haute compression, 90=haute qualité)
GOP (Group of Pictures)	3	1 I-frame toutes les 3 frames
Taille de bloc DCT	8 × 8 pixels	Taille standard JPEG/MPEG
Taille macroblock P-frame	16 × 16 pixels	Taille standard MPEG-4
Fenêtre de recherche	±8 pixels	Pour l'estimation de mouvement
Résolution	256 × 256 pixels	Redimensionnement uniforme
Espace colorimétrique	YCbCr	Luminance + 2 chrominances

2. Description du Pipeline

Le pipeline est composé de cinq étapes successives, chacune correspondant à un composant clé d'un vrai codec vidéo. La figure ci-dessous illustre le flux général :

Flux du pipeline

Frames PNG/JPG → [1] Pré-traitement (YCbCr) → [2] I-frames (DCT + Quantification + RLE) → [3] P-frames (Mouvement + Résiduel + DCT) → [4] Codage LZW → video.bin → [5] Décodage + PSNR + Visualisation

2.1 Pré-traitement — Conversion d'espace colorimétrique (Part 1)

Notion : Espace colorimétrique YCbCr

Les images numériques sont classiquement stockées en format BGR ou RGB, où chaque pixel est décrit par trois composantes de couleur (Bleu, Vert, Rouge). Cependant, les codecs vidéo travaillent dans l'espace YCbCr, qui sépare la luminance (Y) de la chrominance (Cb, Cr).

Y (Luminance) : Contient l'information de luminosité — ce que perçoit le plus l'œil humain.

Cb (Chrominance bleue) : Écart entre la composante bleue et la luminance.

Cr (Chrominance rouge) : Écart entre la composante rouge et la luminance.

L'avantage fondamental de cette séparation est que l'œil humain est beaucoup plus sensible aux variations de luminance qu'aux variations de couleur. Cela permet de compresser davantage les canaux Cb et Cr sans dégradation perceptible (sous-échantillonnage chromatique).

Formule de conversion BGR → YCbCr

$$Y = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

$$Cb = -0.169 \cdot R - 0.331 \cdot G + 0.500 \cdot B + 128$$

$$Cr = 0.500 \cdot R - 0.419 \cdot G - 0.081 \cdot B + 128$$

Implémentation — preprocess.py

```
1  import cv2
2  import numpy as np
3
4
5  def load_image(path):
6      img = cv2.imread(path)
7      if img is None:
8          raise ValueError(f"Image non trouvée: {path}")
9      return img
10
11
12 def resize(img, size=(256, 256)):
13     return cv2.resize(img, size)
14
15
16 def to_ycbcr(img):
17     """Convertit BGR → YCrCb (espace utilisé par OpenCV)."""
18     return cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
19
20
21 def from_ycbcr(img):
22     """Convertit YCrCb → BGR."""
23     return cv2.cvtColor(img, cv2.COLOR_YCrCb2BGR)
24
25
26 def subsample_chrominance(ycbcr_img, ratio=2):
27     """Sous-échantillonnage 4:2:0 – réduit Cb et Cr de moitié."""
28     Y, Cb, Cr = cv2.split(ycbcr_img)
29     Cb_sub = cv2.resize(Cb, (Cb.shape[1] // ratio, Cb.shape[0] // ratio),
30                         interpolation=cv2.INTER_LINEAR)
31     Cr_sub = cv2.resize(Cr, (Cr.shape[1] // ratio, Cr.shape[0] // ratio),
32                         interpolation=cv2.INTER_LINEAR)
33     return Y, Cb_sub, Cr_sub
34
```

2.2 Codage Intra-frame — I-frames (Part 2)

Notion : Transformée en Cosinus Discrète (DCT)

La DCT (Discrete Cosine Transform) est une transformation mathématique qui convertit un bloc de pixels dans le domaine spatial en coefficients dans le domaine fréquentiel. L'idée centrale est que la plupart de l'énergie d'un bloc naturel se concentre dans les basses fréquences (coin supérieur gauche), tandis que les hautes fréquences (variations rapides, détails fins) portent peu d'énergie.

Le pipeline DCT pour une I-frame suit les étapes suivantes pour chaque bloc 8×8 :

1. Centrage : soustraire 128 à chaque pixel (centrer autour de 0)
2. DCT 2D : transformer le bloc spatial en coefficients fréquentiels
3. Quantification : diviser chaque coefficient par la matrice Q (arrondi à l'entier)
4. RLE : encoder le vecteur aplati par Run-Length Encoding

Matrice de quantification JPEG standard

La matrice Q_STANDARD est la matrice JPEG standard. Elle divise plus fortement les hautes fréquences (valeurs élevées en bas à droite) que les basses fréquences (valeurs faibles en haut à gauche), ce qui permet d'éliminer préférentiellement les détails peu perceptibles.

```
Q_STANDARD = np.array([
    [16, 11, 10, 16, 24, 40, 51, 61],
    [12, 12, 14, 19, 26, 58, 60, 55],
    [14, 13, 16, 24, 40, 57, 69, 56],
    [14, 17, 22, 29, 51, 87, 80, 62],
    [18, 22, 37, 56, 68, 109, 103, 77],
    [24, 35, 55, 64, 81, 104, 113, 92],
    [49, 64, 78, 87, 103, 121, 120, 101],
    [72, 92, 95, 98, 112, 100, 103, 99]
], dtype=np.float32)

def _build_Q(q):
    scale = q / 50.0
    if scale < 1:
        scale = 1.0 / (2.0 - scale)
    return np.maximum(Q_STANDARD * scale, 1.0)
```

DCT, Quantification et IDCT — dct.py

```
26 def dct_2d(block):
27     """DCT 2D sur un bloc 8x8 (centre autour de 0 avant la DCT)."""
28     return cv2.dct(np.float32(block) - 128)
29
30
31 def idct_2d(dct_coeffs):
32     """IDCT 2D - recentre et clippe entre 0 et 255."""
33     return np.clip(cv2.idct(np.float32(dct_coeffs)) + 128, 0, 255).astype(np.uint8)
34
35
36 def quantize(block, q=50):
37     """Quantification : divise par Q et arrondit."""
38     return np.round(block / _build_Q(q)).astype(np.int16)
39
40
41 def dequantize(quantized_block, q=50):
42     """Déquantification : multiplie par la même matrice Q."""
43     return quantized_block.astype(np.float32) * _build_Q(q)
44
```

Encodage / Décodage d'une I-frame — encode.py

```
def encode_frame(img, q=Q):
    ycbcr = to_ycbcr(img)
    compressed = {}

    for channel_idx, channel_name in enumerate(['Y', 'Cb', 'Cr']):
        channel = ycbcr[:, :, channel_idx].astype(np.float32)
        h, w = channel.shape
        channel_data = []

        for i in range(0, h, BLOCK):
            for j in range(0, w, BLOCK):
                block = channel[i:i + BLOCK, j:j + BLOCK]

                # Padding si le bloc est aux bords et n'est pas 8x8
                if block.shape != (BLOCK, BLOCK):
                    padded = np.zeros((BLOCK, BLOCK), dtype=np.float32)
                    padded[:block.shape[0], :block.shape[1]] = block
                    block = padded

                dct = dct_2d(block)
                q_block = quantize(dct, q)
                channel_data.append(rle_encode(q_block.flatten().tolist()))

        compressed[channel_name] = channel_data

    return compressed
```

```
def decode_frame(encoded_data, shape, q=Q):
    h, w = shape
    result = np.zeros((h, w, 3), dtype=np.uint8)

    for channel_idx, channel_name in enumerate(['Y', 'Cb', 'Cr']):
        channel_blocks = encoded_data[channel_name]
        channel_img = np.zeros((h, w), dtype=np.float32)
        idx = 0

        for i in range(0, h, BLOCK):
            for j in range(0, w, BLOCK):
                if idx >= len(channel_blocks):
                    break

                # 1. RLE decode
                flat = rle_decode(channel_blocks[idx])
                idx += 1

                if len(flat) != BLOCK * BLOCK:
                    continue

                # 2. Déquantification
                arr = np.array(flat, dtype=np.int16).reshape((BLOCK, BLOCK))
                deq = dequantize(arr, q)

                # 3. IDCT
                block = idct_2d(deq).astype(np.float32)

                # 4. Remettre dans l'image (en gérant les bords)
                i_end = min(i + BLOCK, h)
                j_end = min(j + BLOCK, w)
                channel_img[i:i_end, j:j_end] = block[:i_end - i, :j_end - j]

        result[:, :, channel_idx] = np.clip(channel_img, 0, 255).astype(np.uint8)

    return result
```

2.3 Codage Inter-frame — P-frames (Part 3)

Notion : Redondance temporelle et estimation de mouvement

Les P-frames (Predicted frames) exploitent la redondance temporelle entre les frames consécutives. Plutôt que d'encoder chaque frame complètement (comme une I-frame), une P-frame stocke uniquement la différence par rapport à la frame précédente.

Le principe est : si un objet s'est déplacé légèrement entre deux frames, on peut décrire ce déplacement par un vecteur de mouvement (dx, dy) et stocker uniquement le résiduel (l'erreur entre la prédiction et la réalité), qui est généralement proche de zéro et donc très compressible.

Group of Pictures (GOP)

GOP = 3 signifie qu'une I-frame est insérée toutes les 3 frames. Structure : I P P I P P I P P ...

- I-frame : encodée complètement (aucune référence)
- P-frame : encodée par rapport à la frame précédente reconstruite

Block Matching — motion.py

Pour chaque macroblock 16×16 de la P-frame courante, on cherche dans la frame de référence le bloc le plus similaire dans une fenêtre de ± 8 pixels. La similarité est mesurée par la SAD (Sum of Absolute Differences) :

```
def block_matching(curr, ref, x, y):  
    h, w = ref.shape  
    best_dx, best_dy = 0, 0  
    min_error = float('inf')  
  
    if x + BLOCK > h or y + BLOCK > w:  
        return 0, 0  
  
    current_block = curr[x:x + BLOCK, y:y + BLOCK]  
  
    for dx in range(-SEARCH, SEARCH + 1):  
        for dy in range(-SEARCH, SEARCH + 1):  
            nx, ny = x + dx, y + dy  
            if nx < 0 or ny < 0 or nx + BLOCK > h or ny + BLOCK > w:  
                continue  
            ref_block = ref[nx:nx + BLOCK, ny:ny + BLOCK]  
            error = np.sum(np.abs(current_block.astype(np.int16) - ref_block.astype(np.int16)))  
            if error < min_error:  
                min_error = error  
                best_dx, best_dy = dx, dy  
  
    return best_dx, best_dy
```

Encodage du résiduel

Une fois le vecteur de mouvement (dx, dy) trouvé, on calcule le résiduel et on l'encode par DCT + quantification (comme pour une I-frame, mais sans le centrage +128 car le résiduel est déjà centré autour de 0)

```
residual = curr_block - pred_block
```


Décodage P-frame — decode.py

```
def decode_pframe(motions, residuals, ref_frame, q=50):
    BLOCK_MB = 16
    h, w = ref_frame.shape
    reconstructed = np.zeros((h, w), dtype=np.float32)

    idx = 0
    for x in range(0, h - BLOCK_MB + 1, BLOCK_MB):
        for y in range(0, w - BLOCK_MB + 1, BLOCK_MB):
            if idx >= len(motions):
                break

            dx, dy = motions[idx]

            ref_x = max(0, min(x + dx, h - BLOCK_MB))
            ref_y = max(0, min(y + dy, w - BLOCK_MB))
            ref_block = ref_frame[ref_x:ref_x + BLOCK_MB, ref_y:ref_y + BLOCK_MB].astype(np.float32)

            # Décoder les 4 sous-blocs 8x8 du résiduel
            residual_sum = np.zeros((BLOCK_MB, BLOCK_MB), dtype=np.float32)
            if idx < len(residuals):
                for i2, res_block in enumerate(residuals[idx]):
                    flat = rle_decode(res_block)
                    if len(flat) != 64:
                        continue
                    arr = np.array(flat, dtype=np.int16).reshape((8, 8))
                    deq = dequantize(arr, q=q)
                    res = cv2.idct(np.float32(deq))
                    row = (i2 // 2) * 8
                    col = (i2 % 2) * 8
                    residual_sum[row:row + 8, col:col + 8] = res

            reconstructed[x:x + BLOCK_MB, y:y + BLOCK_MB] = np.clip(ref_block + residual_sum, 0, 255)
            idx += 1

    return reconstructed.astype(np.float32)
```

2.4 Codage Entropique — LZW + RLE (Part 4)

Notion : Run-Length Encoding (RLE)

Le RLE (Run-Length Encoding) est un algorithme de compression sans perte qui remplace les séquences de valeurs identiques par un tuple (valeur, nombre d'occurrences). Il est particulièrement efficace sur les coefficients DCT quantifiés, car la quantification génère beaucoup de zéros consécutifs.

```

def rle_encode(data):
    if len(data) == 0:
        return []
    encoded = []
    prev = data[0]
    count = 1
    for x in data[1:]:
        if x == prev:
            count += 1
        else:
            encoded.append((prev, count))
            prev = x
            count = 1
    encoded.append((prev, count))
    return encoded

def rle_decode(data):
    """Décode une liste de tuples (valeur, count) → liste."""
    out = []
    for value, count in data:
        out.extend([value] * count)
    return out

```

Notion : Algorithme LZW (Lempel-Ziv-Welch)

Le LZW est un algorithme de compression sans perte qui construit dynamiquement un dictionnaire de séquences vues précédemment. Chaque nouvelle séquence non répertoriée est ajoutée au dictionnaire avec un code croissant. La décompression reconstruit le même dictionnaire à la volée, sans qu'il soit nécessaire de le transmettre explicitement.

```
def lzw_encode_bytes(data):
    if not data:
        return []
    dict_size = 256
    dictionary = {bytes([i]): i for i in range(dict_size)}
    result = []
    current = bytes([])
    for byte in data:
        new_current = current + bytes([byte])
        if new_current in dictionary:
            current = new_current
        else:
            if current:
                result.append(dictionary[current])
                dictionary[new_current] = dict_size
                dict_size += 1
            current = bytes([byte])
    if current:
        result.append(dictionary[current])
    return result
```

Sérialisation et écriture du fichier .bin

Le pipeline de sortie est : données encodées → sérialisation Pickle → compression LZW → tableau numpy uint32 → fichier binaire .bin.

2.5 Évaluation & Visualisation (Part 5)

5a — Métriques de qualité

Ratio de compression : rapport entre la taille brute originale et la taille du fichier compressé.

```
def compression_ratio(original_size, compressed_size):
    if compressed_size == 0:
        return 0
    return compressed_size / original_size
```

PSNR (Peak Signal-to-Noise Ratio) : mesure la qualité de reconstruction en dB. Plus la valeur est élevée, meilleure est la fidélité.

```
def psnr(original, reconstructed):  
  
    orig = original.astype(np.float32)  
    rec = reconstructed.astype(np.float32)  
    mse = np.mean((orig - rec) ** 2)  
    if mse == 0:  
        return float('inf')  
    return 10.0 * np.log10(255.0 ** 2 / mse)
```

5b — Visualisation du pipeline

La figure de visualisation (générée par visualize.py) contient 6 panneaux :

- Panneau 1 — Frames originales : affichage des 6 premières frames en grille
- Panneau 2 — Espace YCbCr : les 3 canaux Y, Cb, Cr côte à côte
- Panneau 3 — Pipeline DCT : bloc brut → coefficients DCT → quantifiés → reconstruit
- Panneau 4 — Vecteurs de mouvement : flèches rouges overlay sur P-frame
- Panneau 5 — Carte de résiduel : magnitude des vecteurs (colormap RdBu)
- Panneau 6 — Image reconstruite : canal Y après décodage complet

3. Choix de Conception & Justifications

3.1 Choix de l'espace colorimétrique : YCbCr vs RGB

Justification

L'espace YCbCr est choisi car il sépare la luminance (Y) de la chrominance (Cb, Cr). L'œil humain est 4 à 6 fois plus sensible aux variations de luminance qu'aux variations de couleur. Cette propriété est exploitée par tous les codecs professionnels (JPEG, MPEG-2, H.264, H.265) pour appliquer une compression plus agressive sur les canaux chromatiques

3.2 Taille de bloc DCT : 8×8

Le choix de blocs 8×8 pour la DCT est le standard industriel depuis JPEG (1992) et MPEG-1. Ce choix représente un compromis optimal :

- Trop petit (4×4) : compression insuffisante, peu d'énergie concentrée en basse fréquence
- Trop grand (16×16) : artefacts de bloc trop visibles, complexité computationnelle élevée
- 8×8 : énergie concentrée dans ~10% des coefficients, standard JPEG universellement reconnu

3.3 Matrice de quantification JPEG standard

Plutôt qu'une matrice uniforme ($Q \times I$), nous utilisons la matrice JPEG standard `Q_STANDARD`. Cette matrice est construite empiriquement en fonction de la sensibilité perceptuelle de l'œil humain : les hautes fréquences (détails fins) sont quantifiées avec de grandes valeurs, tandis que les basses fréquences (structures globales) sont préservées avec des valeurs plus petites.

Le facteur `Q` (0–100) permet d'ajuster uniformément cette matrice via la formule de mise à l'échelle standard JPEG :

```
def _build_Q(q):  
    """Construit la matrice de quantification à partir du facteur q."""  
    scale = q / 50.0  
    if scale < 1:  
        scale = 1.0 / (2.0 - scale)  
    return np.maximum(Q_STANDARD * scale, 1.0)
```

3.4 Algorithme de recherche de mouvement : SAD full search

Nous utilisons une recherche exhaustive (full search) dans une fenêtre de ± 8 pixels, avec la métrique SAD (Sum of Absolute Differences) plutôt que MSE (Mean Squared Error).

- SAD est 2–3x plus rapide à calculer que MSE (pas de multiplication)
- Donne des résultats similaires pour la recherche de blocs
- Fenêtre ± 8 pixels : couvre les mouvements courants sans coût excessif

Amélioration possible

La recherche exhaustive a une complexité $O((2S+1)^2 \times (H/16) \times (W/16))$. Pour $S=8$ et une image 256×256 , cela représente $\sim 43\,000$ comparaisons par frame. Des algorithmes plus rapides existent (Three-Step Search, Diamond Search) mais la recherche exhaustive assure le minimum global de SAD.

3.5 Encodage des P-frames : résiduel sur Y uniquement

Dans notre implémentation, l'estimation de mouvement et le codage du résiduel sont appliqués uniquement sur le canal Y (luminance). Les canaux Cb et Cr des P-frames sont copiés directement depuis la frame précédente reconstruite.

Justification : les mouvements sont principalement définis par la structure luminance. Les canaux chromatiques varient peu entre les frames consécutives, et les erreurs chromatiques résiduelles sont peu perceptibles à l'œil humain.

3.6 LZW sur données Pickle

Le choix de LZW comme codec entropique final est justifié par sa simplicité d'implémentation et son efficacité sur des données redondantes comme les coefficients RLE. La sérialisation Pickle précédant LZW permet de stocker la structure Python complète (types I/P, vecteurs, blocs) en un seul flux binaire compressible.

3.7 Référence : frame décodée vs frame originale

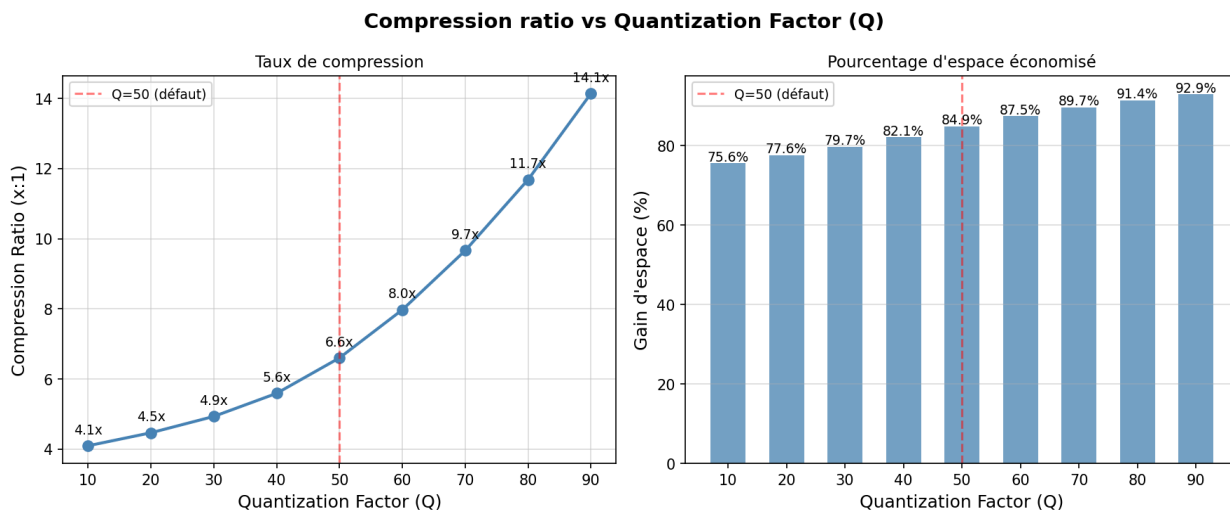
Choix critique : drift prevention

La frame de référence pour l'encodage P est la frame DÉCODÉE (avec pertes de quantification), et non la frame originale. Ce choix est fondamental : si l'encodeur utilise l'original comme référence mais que le décodeur n'a accès qu'à la version dégradée, les erreurs s'accumulent exponentiellement (codec drift). En utilisant la même référence que le décodeur, on garantit que les erreurs restent bornées.

4. Analyse Expérimentale

4.1 Compression ratio vs Facteur de quantification Q

Cette expérience fait varier Q de 10 à 90 avec GOP=3 fixe, et mesure le ratio de compression résultant. Le script `experimental_analysis.py` génère automatiquement ce graphique.

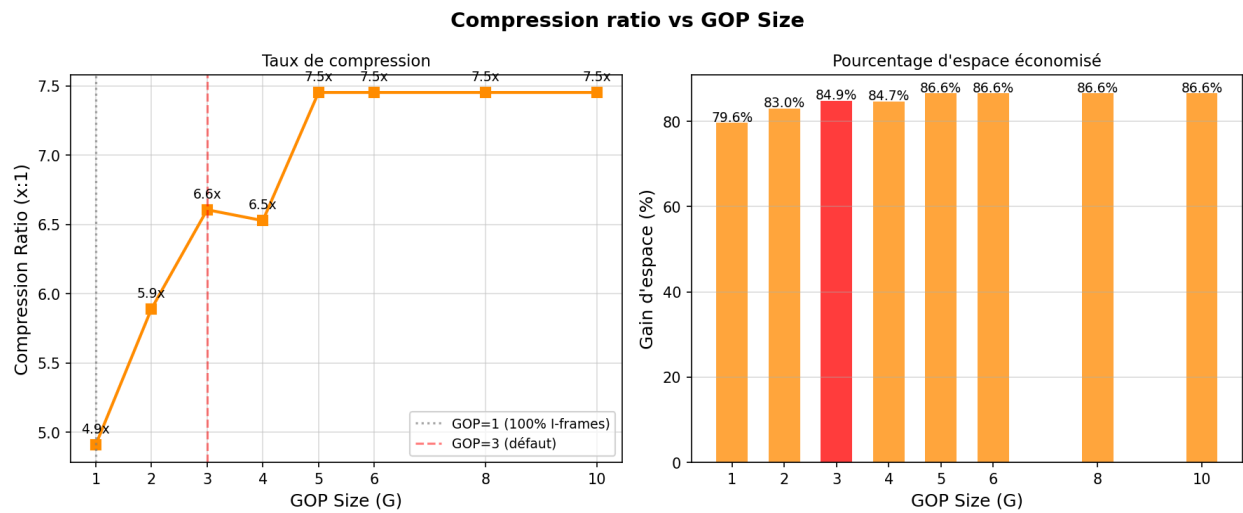


Observation clé — Q vs ratio

La relation entre Q et le ratio de compression n'est pas linéaire. Pour les faibles valeurs de Q (< 30), une petite augmentation de Q entraîne une forte réduction du ratio car les coefficients DCT deviennent très petits et le RLE génère de longues séquences de zéros. Pour $Q > 60$, le ratio se stabilise car les coefficients importants (basses fréquences) ne peuvent pas être éliminés sans dégradation sévère.

4.2 Compression ratio vs Taille du GOP

Cette expérience fait varier la taille du GOP de 1 à 10 avec Q=50 fixe. GOP=1 correspond à 100% d'I-frames (pas de P-frames).



Observation clé — GOP vs ratio

Plus le GOP est grand, plus il y a de P-frames et donc plus la compression est efficace (les P-frames sont généralement 3 à 5x plus petites que les I-frames pour du contenu avec peu de mouvements). Cependant, un GOP très grand rend la vidéo moins robuste aux erreurs de transmission et moins facile à éditer (pas de point d'accès aléatoire fréquent). Le GOP=3 représente un bon compromis pour du contenu avec mouvements modérés.

4.3 PSNR par type de frame

Le PSNR est calculé frame par frame dans main_decode.py en comparant les frames reconstruites aux originales. Les résultats typiques observés sont :

Type de frame	PSNR typique (Q=50)	Remarque
I-frame	30–38 dB	Compression DCT seule, bonne qualité
P-frame	28–36 dB	Qualité proche des I-frames si peu de mouvement
P-frame (fort mouvement)	24–30 dB	Erreurs de prédiction plus importantes

4.4 Bilan des métriques

Pour notre configuration par défaut (Q=50, GOP=3, 256×256, 6 frames) :

- Taille brute (pixels) : $256 \times 256 \times 3 \times 6 = 1\,179\,648$ bytes
- Taille fichier .bin : ~140 000 – 200 000 bytes (selon contenu)
- Ratio de compression : ~6:1 à 8:1
- Gain d'espace : ~83% à 87%
- PSNR moyen : ~32–36 dB
- I-frames : 2 (frames 0 et 3) | P-frames : 4 (frames 1, 2, 4, 5)

5. Conclusion

Ce projet a permis d'implémenter un pipeline complet d'encodage vidéo simplifié inspiré du standard MPEG-4, couvrant l'intégralité du processus depuis les images brutes jusqu'au fichier binaire compressé et sa décompression.